

```

"""
dataset.py - PyTorch dataset, normalisation, and dataloaders.
"""

from __future__ import annotations

from pathlib import Path
from typing import Dict, Optional, Tuple

import numpy as np
import pandas as pd
import torch
from torch.utils.data import DataLoader, Dataset

from config import (
    PARAM_BOUNDS, PARAM_NAMES, STATE_NAMES,
    DT, T_IGNORE, TRAIN_CFG,
)

# -----
# Normalisation statistics (saved / loaded as .npz)
# -----

class NormStats:
    """Holds mean/std for road, state, accel + lo/hi for params."""

    def __init__(self,
                 r_mean: np.ndarray, r_std: np.ndarray,
                 s_mean: np.ndarray, s_std: np.ndarray,
                 a_mean: np.ndarray, a_std: np.ndarray,
                 p_lo: np.ndarray, p_hi: np.ndarray):
        self.r_mean = r_mean.astype(np.float32)
        self.r_std = r_std.astype(np.float32)
        self.s_mean = s_mean.astype(np.float32)
        self.s_std = s_std.astype(np.float32)
        self.a_mean = a_mean.astype(np.float32)
        self.a_std = a_std.astype(np.float32)
        self.p_lo = p_lo.astype(np.float32)
        self.p_hi = p_hi.astype(np.float32)

    def save(self, path: str) -> None:
        np.savez(path, r_mean=self.r_mean, r_std=self.r_std,
                 s_mean=self.s_mean, s_std=self.s_std,
                 a_mean=self.a_mean, a_std=self.a_std,

```

```

        p_lo=self.p_lo,        p_hi=self.p_hi)
    print(f"Normalisation stats saved → {path}")

    @staticmethod
    def load(path: str) -> "NormStats":
        d = np.load(path)
        return NormStats(**{k: d[k] for k in d.files})

    @staticmethod
    def from_dataframe(df: pd.DataFrame) -> "NormStats":
        """Compute stats from the *training* subset of a DataFrame."""
        road_cols = ["z_1f", "ph_f", "z_2", "ph_2", "z_3", "ph_3"]
        state_cols = STATE_NAMES
        accel_cols = [f"qdd_{n}" for n in STATE_NAMES[:3]]

        if all(c in df.columns for c in road_cols):
            r_arr = df[road_cols].values.astype(np.float32)
        else:
            r_arr = np.zeros((1, 6), np.float32)

        s_arr = df[state_cols].values.astype(np.float32)
        a_arr = df[accel_cols].values.astype(np.float32)

        def ms(arr):
            return arr.mean(0), arr.std(0) + 1e-8

        r_mean, r_std = ms(r_arr)
        s_mean, s_std = ms(s_arr)
        # Accelerations: normalise by std only, keep mean=0.
        # This preserves zero-crossings and AC content which the LSTM must learn.
        # Subtracting a non-zero mean would shift the signal and destroy
        # the RMS information the surrogate needs to capture.
        a_mean = np.zeros(a_arr.shape[1], np.float32)
        _, a_std = ms(a_arr)
        p_lo = np.array([PARAM_BOUNDS[k][0] for k in PARAM_NAMES], np.float32)
        p_hi = np.array([PARAM_BOUNDS[k][1] for k in PARAM_NAMES], np.float32)
        return NormStats(r_mean, r_std, s_mean, s_std, a_mean, a_std, p_lo, p_hi)

# -----
# Dataset
# -----

ROAD_COLS = ["z_1f", "ph_f", "z_2", "ph_2", "z_3", "ph_3"]
ACCEL_COLS = [f"qdd_{n}" for n in STATE_NAMES[:3]] # z_c, th_c, ph_c

```

```

class CabinDataset(Dataset):
    """
    One sample = one full ODE case (time series).

    Returns dict with keys:
        road    : [T, 6]    normalised axle inputs
        param   : [8]       normalised design parameters
        state   : [T, 6]    normalised states
        accel   : [T, 3]    normalised accels (z_c, th_c, ph_c)
        rms     : [1]       cabin seat RMS in physical units [m/s2]
        case_id: int
    """

    def __init__(self,
                  csv_path: str,
                  norm_stats: NormStats,
                  split: str = "train", # "train" | "test" | "all"
                  h_seat: float = 0.6, # kept for API compat, not used
                  t_ignore: float = T_IGNORE,
                  road_arr: Optional[np.ndarray] = None, # [T, 6] pre-computed
                  t_eval: Optional[np.ndarray] = None,
                  cfg: dict = None): # vehicle config for hcp
        df = pd.read_csv(csv_path)

        if split != "all" and "split" in df.columns:
            df = df[df["split"] == split].copy()

        self.ns = norm_stats
        self.h_seat = h_seat
        self.t_ignore = t_ignore
        self.cfg = cfg or {}
        self.samples = []

        for cid, grp in df.groupby("case_id"):
            grp = grp.sort_values("t").reset_index(drop=True)
            T = len(grp)

            # — road inputs —————
            if all(c in grp.columns for c in ROAD_COLS):
                road_raw = grp[ROAD_COLS].values.astype(np.float32)
            elif road_arr is not None:
                road_raw = road_arr[:T].astype(np.float32)
            else:
                road_raw = np.zeros((T, 6), np.float32)
            road_norm = (road_raw - norm_stats.r_mean) / norm_stats.r_std

            # — parameters —————

```

```

p_raw = np.array([float(grp[k].iloc[0]) for k in PARAM_NAMES],
                  np.float32)
p_norm = (p_raw - norm_stats.p_lo) / (norm_stats.p_hi - norm_stats.p_lo)

# — states & accels —————
s_raw = grp[STATE_NAMES].values.astype(np.float32)
a_raw = grp[ACCEL_COLS].values.astype(np.float32)
s_norm = (s_raw - norm_stats.s_mean) / norm_stats.s_std
a_norm = (a_raw - norm_stats.a_mean) / norm_stats.a_std

# — seat RMS: ISO 2631 3-axis combined —————
# az = qdd_z_c (index 0)
# ax = -hcp * qdd_th_c (index 1, pitch)
# ay = hcp * qdd_ph_c (index 2, roll)
# RMS = sqrt(mean(az2) + mean(ax2) + mean(ay2))
t_vals = grp["t"].values
mask = t_vals >= t_ignore
if mask.any():
    h_cp = float(self.cfg.get("hcp", 0.1))
    az = a_raw[mask, 0]
    ax = -h_cp * a_raw[mask, 1]
    ay = h_cp * a_raw[mask, 2]
    rms_val = float(np.sqrt(
        np.mean(az**2) + np.mean(ax**2) + np.mean(ay**2)))
else:
    rms_val = 0.0

# Store a_std so the loss can reconstruct physical RMS
a_std_stored = norm_stats.a_std # [3]

self.samples.append({
    "road": torch.tensor(road_norm, dtype=torch.float32),
    "param": torch.tensor(p_norm, dtype=torch.float32),
    "state": torch.tensor(s_norm, dtype=torch.float32),
    "accel": torch.tensor(a_norm, dtype=torch.float32),
    "accel_raw": torch.tensor(a_raw, dtype=torch.float32),
    "rms": torch.tensor([rms_val], dtype=torch.float32),
    "a_std": torch.tensor(a_std_stored, dtype=torch.float32),
    "case_id": int(cid),
})

def __len__(self) -> int: return len(self.samples)
def __getitem__(self, i): return self.samples[i]

# —————
# Collate (pads variable-length sequences within a batch)

```

```

# -----

def collate_pad(batch):
    """Pad road / state / accel tensors to same T in batch."""
    max_T = max(s["road"].shape[0] for s in batch)
    out = {k: [] for k in batch[0] if k != "case_id"}
    out["case_id"] = [s["case_id"] for s in batch]

    pad_keys = {"road", "state", "accel", "accel_raw"}
    nopad_keys = {"param", "rms", "a_std"}

    for s in batch:
        T = s["road"].shape[0]
        pad = max_T - T
        for k in pad_keys:
            if k in s:
                out[k].append(
                    torch.nn.functional.pad(s[k], (0, 0, 0, pad)))
        for k in nopad_keys:
            if k in s:
                out[k].append(s[k])

    return {k: (torch.stack(v) if k != "case_id" else v)
            for k, v in out.items()}

# -----
# Build loaders
# -----

def build_loaders(
    csv_path: str,
    norm_stats: NormStats,
    batch_size: int = TRAIN_CFG["batch_size"],
    h_sea: float = 0.6,
    road_arr: Optional[np.ndarray] = None,
    num_workers: int = 0,
    cfg: dict = None,
) -> Tuple[DataLoader, DataLoader, DataLoader]:
    """
    Returns (train_loader, val_loader, test_loader).
    Train/val split is within the 'train' split rows (80/20).
    Test loader uses 'test' split rows.
    Handles tiny datasets gracefully (e.g. smoke-test with 2 cases).
    """
    # — full train dataset -----
    train_ds = CabinDataset(csv_path, norm_stats, split="train",

```

```

                                h_seat=h_seat, road_arr=road_arr, cfg=cfg)
n_total = len(train_ds)

# — train / val split —————
# Need at least 1 sample in each split.
# For tiny datasets (smoke test) we allow val to overlap train
# rather than crashing.
if n_total >= 2:
    n_val    = max(1, n_total // 5)
    n_train  = n_total - n_val
    train_sub, val_sub = torch.utils.data.random_split(
        train_ds, [n_train, n_val],
        generator=torch.Generator().manual_seed(0)
    )
else:
    # Only 1 training case: use it for both train and val
    n_train = n_total
    n_val    = n_total
    train_sub = train_ds
    val_sub   = train_ds

# — test dataset —————
test_ds = CabinDataset(csv_path, norm_stats, split="test",
                        h_seat=h_seat, road_arr=road_arr, cfg=cfg)

# If no test cases exist, fall back to using train data
# (only happens in extreme smoke-test with 1 total case)
if len(test_ds) == 0:
    test_ds = train_ds

# — cap batch_size to dataset size to avoid empty batches —
eff_bs_train = min(batch_size, n_train)
eff_bs_val    = min(batch_size, n_val)
eff_bs_test   = min(batch_size, max(1, len(test_ds)))

kw = dict(collate_fn=collate_pad, num_workers=num_workers,
          pin_memory=torch.cuda.is_available())

train_ld = DataLoader(train_sub, batch_size=eff_bs_train,
                       shuffle=True, **kw)
val_ld    = DataLoader(val_sub, batch_size=eff_bs_val,
                       shuffle=False, **kw)
test_ld   = DataLoader(test_ds, batch_size=eff_bs_test,
                       shuffle=False, **kw)

print(f"Dataset splits: train={n_train} val={n_val} "
      f"test={len(test_ds)} cases ")

```

```
        f"(batch_size train={eff_bs_train} val={eff_bs_val})")
    return train_ld, val_ld, test_ld
```